CPSC 121 - Models of Computation

Lab 04 Notes

These notes are intended to be a guide through the labs in this course. While the lab documents walk you through specific exercises, it can often be difficult to know what to focus on; these notes will provide the following to help guide your studying:

- learning objectives (the content that is testable!),
- review notes providing details on the testable content from each lab, and
- practice questions for each lab.

None of this content is new - it should all be covered in your lab's exercises. Therefore, this is a supplementary document and not any sort of required reading. **This document is just extra if you find it helpful :)**

Note that these practice questions may not be in the style of exam questions, but should help you test your knowledge on testable content. Also note that we won't ask you to work with ICs on the exam, but they're the direct application of the digital logic you're studying in the labs and we obviously want you to care about the physical application! Note: When labs have multiple topics, we tend to focus the questions on lab-exclusive topics (such as MUXs), that you've seen less in class. These questions focus more on circuits, as the non-circuit content is often covered in lecture, tutorial, and assignments.

These notes are being written as we go through the course so please be patient with us as we release them. **As these notes are new, notify us on Piazza if you find any typos/issues with the content. If you have suggestions for how to improve the notes, please let us know as well!**

1 Computation

In this lab, we explore how to construct circuits that can perform various computations.

2 Learning Objectives

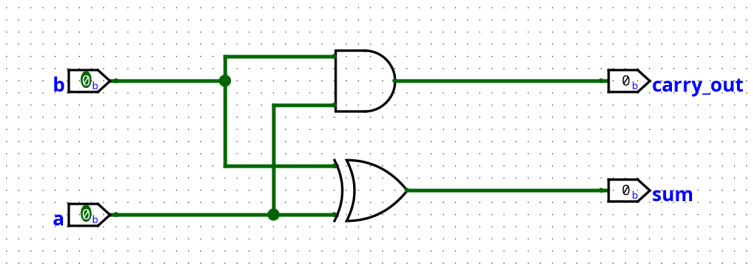
- Be able to analyze and construct circuits for basic computation (i.e., half-adders/adders and larger circuits using half-adders/adders).
- Understand the purpose of a "carry bit" and overflow/underflow in the context of computation.
- Understand a basic model for how a computer may perform simple multiplication or division through subtraction and addition.
- Understand and discuss the purpose of the ALU module in a computer.
- Understand and discuss the purpose of the RAM module in a computer.

3 Review Notes

3.1 Adders

In the pre-lab, we make students create a half-adder and full-adder.

A half-adder adds two values a and b together to output a sum and carry bit:



a	b	carry	sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Figure 1: Half-Adder and Truth Table

A full-adder works similarly to half-adders but also takes in a carry in. It computes the following: $sum \equiv (a \oplus b) \oplus carry_in$, and $carry_out \equiv (a \wedge b) \oplus (carry_in \wedge (a \oplus b))$.

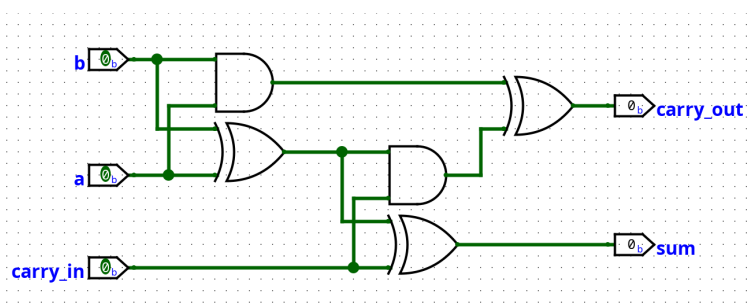


Figure 2: Full-Adder

3.2 Multipliers/Dividers and Memory

These two components are fundamental to performing computation, as you can easily build on them to compute more complex things. As we discuss in the further analysis TODO, we can create a multiplier module that is time-dependent. Every cycle, the circuit will add once again to the result (as multiplication is simply repeated addition). This uses the following components:

- An adder module
- A memory unit, to keep track of your result of the addition (to be discussed in Lab 5)
- A counter, to ensure your computation stops after the correct number of additions (to be discussed in Lab 5/6)

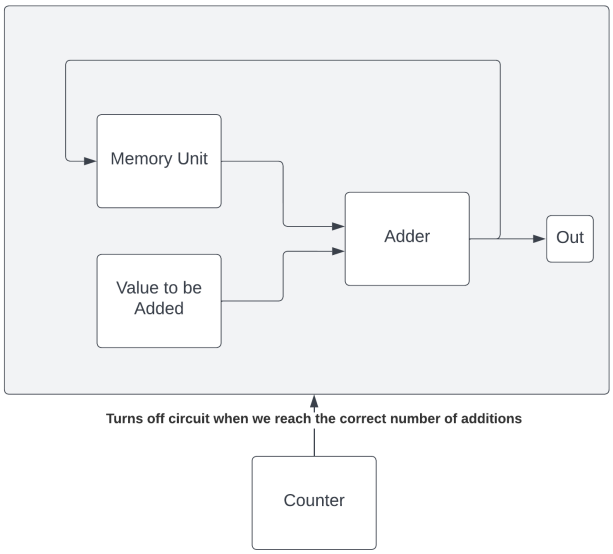


Figure 3: Sketch of Multiplier Circuit

Similarly, you can create a divider using a subtractor module. The main difference is that the output of a divison (the quotient) is the number of times the subtraction can be performed. Therefore, your counter will keep track of the result (adding one at each iteration).

- A subtractor module
- A memory unit, to keep track of your result of the subtraction (to be discussed in Lab 5)
- A counter, to keep track of how many times we've performed subtraction (to be discussed in Lab 5/6)

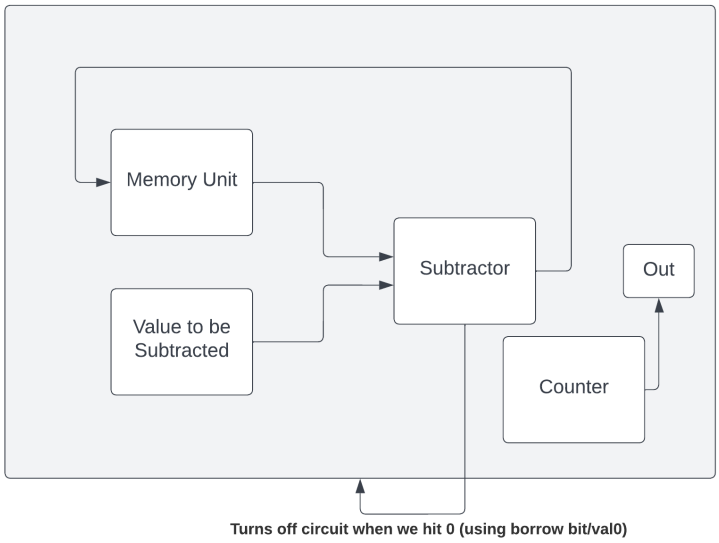


Figure 4: Sketch of Subtractor Circuit

For the above topics, we mentioned a "carry bit". Say you do the binary operation **1** + **1** - this would equal to **10**. To capture our full output, our adder requires two output bits: **1** and **0**. Here, we call the **1** a **"carry bit"**, as it is carried to the next position, in front of the 0. In circuits, modules such as adders, subtractors (when you go below 0, we have a "borrow" bit), and others often have various carry bits like these. This lets you capture all your information from the computation, in order to input it to the rest of the circuit. As we discussed in lab, this is key for our ALU (in the next section!) to know when to start/stop computation, and these carry bits can represent overflow, underflow, and other crucial information.

3.3 A Working Computer - ALU and RAM

We also started discussing the first components of a computer: RAM and ALU.

- The Random-Access Memory (or RAM) module temporarily stores code and data. Programs store intermediate values there and things that they're actively working on. As you saw, the computer will store various hexadecimal instructions there that will be retrieved later.

- The Arithmetic Logic Unit (or ALU) module works on computation. It performs addition, subtraction, multiplication, and more mathematical operations. It also performs **logical** computation, for use in decision making. In this lab, we've shown you a much simplified ALU.